

COMPILATION & CONTRIBUTING GUIDE

This guide provides instructions for developers compiling Malkhana Vault from source, setting up the local environment, and contributing changes.

1. Prerequisites

Malkhana Vault compiles as a Tauri v2 application, requiring a hybrid Node.js frontend and Rust compiler environment.

Windows (Compilation Host)

- **Node.js:** v18.0 or later (includes `npm`).
- **Rust:** Stable toolchain (`rustc` and `cargo` v1.75+ or v1.95+).
- **C++ Compiler:** Visual Studio Build Tools (C++ Build Tools workload).
- **Git:** Installed and added to path.

Linux / Debian Target

- **WebKitGTK:** Required for Tauri WebView rendering.

```
sudo apt install libwebkit2gtk-4.1-dev build-essential curl wget file libssl-dev libgtk-3-dev  
libayatana-appindicator3-dev librsvg2-dev
```

2. SQLCipher Compilation Notes

Malkhana Vault secures local SQLite databases using SQLCipher (AES-256-CBC encryption).

- **Static Bundling:** The Rust backend (`cargo.toml`) compiles `rusqlite` with the `bundled-sqlcipher` feature flag.
- **Why?** This automatically builds SQLCipher from source and links it statically into the executable. This eliminates the need to compile or install OpenSSL or SQLCipher dlls separately on target workstations, ensuring portability for police deployments.
- **Prerequisites:** Statically compiling SQLCipher requires a valid C compiler (MSVC on Windows, GCC/Clang on Linux) to be available during `cargo build`.

3. Step-by-Step Build Instructions

Follow these steps to run and build the application locally:

```
# 1. Clone the repository  
git clone https://github.com/chandranshgupta/Malkhana.git  
cd Malkhana  
  
# 2. Install Node.js dependencies  
npm install  
  
# 3. Launch the development server (runs Vite dev server + cargo watch)  
npm run tauri:dev  
  
# 4. Compile the production package (AppImage, .deb, or .exe depending on host OS)  
npm run tauri:build
```

4. Biometric Witness Mock Implementation

To enforce the **Step 0 Login Custody** workflow without requiring active fingerprint scanners, microphones, or specialized cameras during developer testing:

- **Active Mock State:** Biometric capture (webcam snapshot and audio sample recording) is currently simulated using a mock capture sequence in `src-tauri/src/commands/session_commands.rs`.
- **Placeholder Hashing:** The frontend passes a static mock string, and the backend generates a valid SHA-256 hash of this payload to verify database insertion and Merkle-tree validation.
- **Hardware Integration Roadmap:** These mock endpoints are designed to be replaced by vendor-specific SDK integrations (e.g. fingerprint scanner APIs or native audio/video capture devices) without modifying the database schema or the frontend state flow.

CHANDRANSH GUPTA — FORENSIC CUSTODY SEAL